

CAPSTONE WORKSHOP

สร้างเว็บจองห้องจาก Requirement

เอกสารนี้คือโจทย์ที่ใช้ทำ Workshop ผู้สอนจะไม่แจกแบบหน้าจอ โครงสร้างตาราง หรือขั้นตอนสร้างสำเร็จรูป ผู้เรียนต้องอ่านโจทย์ วางแผน และตัดสินใจออกแบบระบบเอง

โจทย์

มหาวิทยาลัยต้องการเว็บสำหรับจองห้องอ่านหนังสือ
ผู้ใช้งานต้องเห็นห้องและช่วงเวลาที่ไม่ว่าง จองห้องได้
และจัดการเฉพาะการจองของตนเอง

เวลา

110 นาที

วางแผน สร้าง ทดสอบ และ deploy

รูปแบบ

ทำคนละ 1 ระบบ

ใช้ Requirement เดียวกัน
แต่หน้าตาและโครงสร้างออกแบบเอง

Stack

Next.js + Supabase

TypeScript, Tailwind, PostgreSQL
และ Vercel

เป้าหมายของงาน

- แปลง Requirement เป็น implementation plan ก่อนเริ่มแก้ไฟล์
- ใช้ Codex ช่วยทำงานโดยกำหนดขอบเขต ตรวจสอบ diff และดูผลการทดสอบทุกครั้ง
- ออกแบบข้อมูลและสิทธิ์ให้ฐานข้อมูลช่วยบังคับกฏของระบบ
- เปิดเว็บจาก production URL และแสดงหลักฐานว่า Acceptance Tests ที่สำคัญผ่าน

เริ่มจาก Plan

ให้ Codex อ่านโปรเจกต์และ Requirement โดยยังไม่แก้ไฟล์ จากนั้นตรวจสอบ routes, components, schema, constraints, RLS และ test plan ก่อนอนุญาตให้ลงมือทำ

01 FUNCTIONAL REQUIREMENTS

ระบบต้องทำอะไรได้บ้าง

Requirement ทุกข้อคือพฤติกรรมที่ผู้ใช้ต้องเห็นหรือทดสอบได้ รูปแบบหน้าจอและจำนวนหน้ายังเป็นสิ่งที่ผู้เรียนตัดสินใจเอง

- 1** บัญชีผู้ใช้
สมัครสมาชิก เข้าสู่ระบบ และออกจากระบบด้วย email/password ได้
- 2** การป้องกันหน้า
ผู้ที่ยังไม่เข้าสู่ระบบเปิดหน้าจอไม่ได้ และถูกนำไปหน้าเข้าสู่ระบบ
- 3** รายชื่อห้อง
อ่านรายชื่อห้องจาก Supabase ห้ามใช้ array ที่เขียนค้างไว้ใน source code
- 4** ช่วงเวลาที่ไม่ว่าง
ผู้ใช้เห็นรายการจองหรือสถานะช่วงเวลาที่ถูกจองแล้วก่อนสร้างรายการใหม่
- 5** สร้างการจอง
เลือกห้อง วันที่ รอบเวลา และกรอกวัตถุประสงค์ของการใช้งาน
- 6** จัดการรายการของตนเอง
แก้ไขและยกเลิกเฉพาะการจองที่ตนเองเป็นเจ้าของ
- 7** ป้องกันการจองซ้ำ
ห้องเดียวกัน วันที่เดียวกัน และรอบเดียวกันมีได้เพียงหนึ่งการจอง
- 8** ข้อมูลคงอยู่
รายการที่บันทึกแล้วยังอยู่หลัง refresh หรือเปิดเว็บใหม่
- 9** สถานะของหน้าจอ
มี loading, empty, validation และ error state ที่ผู้ใช้เข้าใจได้
- 10** ใช้งานบน production
เว็บทำงานผ่าน Vercel URL โดยไม่ต้องเปิด localhost

02 DATA AND SECURITY

กติกาสที่ต้องบังคับในระบบ

การขออนุมัติในหน้าเว็บช่วยเรื่อง UX แต่ไม่ถือว่าการป้องกันข้อมูล สิทธิ์ต้องถูกตรวจที่ Supabase และ PostgreSQL ด้วย

ข้อกำหนดด้านความปลอดภัย

- ใช้ Supabase Email/Password Authentication
- อ่านตัวตนของผู้ใช้จาก session ห้ามรับ user_id จากฟอร์มแล้วเชิ้อทันที
- เปิดใช้ Row Level Security (RLS) กับตารางที่มีข้อมูลผู้ใช้
- ผู้ใช้ที่เข้าสู่ระบบอ่านข้อมูลที่เป็นต่อการดูช่วงเวลาว่างได้ แต่แก้ไขและลบได้เฉพาะรายการของตนเอง
- สร้าง database constraint เพื่อป้องกัน room/date/slot ซ้ำ แม้มี request เข้ามาพร้อมกัน
- ห้ามใส่ secret key, legacy service_role key หรือข้อมูลลับใน frontend และ Git repository

Auth ไม่ใช่ RLS

Authentication ตอบว่าใครกำลังใช้งาน ส่วน Authorization และ RLS
ตอบว่าผู้ใช้นั้นอ่านหรือแก้ไข row ไດได้

ข้อกำหนดทางเทคนิค

ส่วน	ข้อกำหนด
Web	Next.js App Router, TypeScript และ Tailwind CSS
Data	Supabase PostgreSQL พร้อม primary key, foreign key, data types และ constraints ที่เหมาะสม
Auth	Supabase Auth และ session ที่ทำงานทั้ง local และ production
Rendering	ใช้ Server Components สำหรับอ่านข้อมูลเมื่อเหมาะสม
Mutation	ใช้ Server Actions สำหรับ Auth และการเปลี่ยนข้อมูล
Source control	ใช้ GitHub, npm และเก็บ package-lock.json ใน repository
Deployment	Deploy Next.js บน Vercel และตั้ง environment variables กับ auth redirect URL ให้ครบ

03 DESIGN SPACE

ส่วนที่ออกแบบเอง และสิ่งที่ไม่ต้องทำ

ทุกคนได้รับโจทย์เดียวกัน แต่ไม่จำเป็นต้องได้เว็บหน้าตาเหมือนกัน การตัดสินใจต้องอธิบายได้และไม่ขัดกับ Requirement

ออกแบบเอง

- Layout, สี, typography และ visual style
- จำนวนหน้าและรูปแบบ navigation
- ใช้ card, table, calendar หรือ form แบบใด
- การแบ่ง component และลำดับการใช้งาน
- ชื่อตารางและ column
- รูปแบบ loading, empty และ error UI
- feature เพิ่มเติมหลัง Requirement หลักผ่านแล้ว

ไม่อยู่ในขอบเขต

- Admin dashboard
- Payment
- Email notification จริง
- File upload หรือ Supabase Storage
- Social login
- การเลือกเวลาเริ่มและสิ้นสุดแบบอิสระ
- Production-grade email infrastructure

สิ่งที่ต้องมีใน implementation plan

- Routes และหน้าหลักของระบบ
- Components และหน้าที่ของแต่ละส่วน
- Tables, columns, relationships และ constraints
- RLS policies ที่ต้องมี พร้อมเหตุผลว่าใครทำอะไรได้
- ลำดับ implementation ที่ลดความเสี่ยงและตรวจได้ที่ละส่วน
- Test cases ที่ผูกกับ Acceptance Tests ในหน้าถัดไป

ก่อนให้ Codex implement

ตรวจสอบว่าแผนครบ Requirement และไม่มีงานนอกขอบเขต หากยังอธิบาย schema หรือ policy ไม่ได้ ให้แก้แผนก่อนเริ่มเขียนโค้ด

04 ACCEPTANCE TESTS

เกณฑ์ตรวจว่างานเสร็จจริง

การที่ Codex บอกว่าเสร็จแล้วไม่ใช่หลักฐาน ต้องทดลองกับเว็บและฐานข้อมูลตามรายการนี้

01 สมัครและเข้าสู่ระบบด้วย email/password ได้	02 ผู้ที่ยังไม่เข้าสู่ระบบถูกนำไปหน้า login
03 ห้องถูกอ่านจาก Supabase ไม่ใช่ hard-coded array	04 สร้าง booking แล้วข้อมูลยังอยู่หลัง refresh
05 แก้ไขและลบ booking ของตนเองได้	06 ผู้ใช้คนที่สองแก้ไข booking ของผู้ใช้งานแรกไม่ได้
07 room/date/slot เข้าถูก database ปฏิเสธ	08 ฟอรัมที่ข้อมูลไม่ควรถูกปฏิเสธ
09 Database error ถูกแปลงเป็นข้อความที่ผู้ใช้เข้าใจได้	10 Repository ไม่มี secret หรือ service-role key
11 Production URL ทำงานโดยไม่พึ่ง localhost	12 Login callback และ environment variables ทำงานบน Vercel

สิ่งที่ส่ง

- GitHub repository พร้อม README.md ที่อธิบายวิธีรันและภาพรวม architecture
- Vercel production URL
- AI_WORKLOG.md ที่มี planning prompt, implementation prompt, debug prompt และ final review prompt
- ER diagram หรือคำอธิบาย schema และ relationships
- Self-test checklist พร้อมระบุข้อที่ผ่าน ข้อที่ยังไม่ผ่าน และปัญหาที่ยังแก้ไม่ได้

ส่งตามสถานะจริง

ช่อง GitHub และ Vercel ในแบบส่งงานไม่บังคับ หากยังทำไม่เสร็จ ให้ส่งข้อมูลที่มีและอธิบายปัญหา ห้ามใส่ลิงก์ปลอม ห้ามส่ง password, secret key หรือข้อมูลบัญชี